Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

TrackSeed: Augmented Reality Cone Placement and a Track Digital Twin for Formula Student – State of the Art and Project Proposal

André Lopes, Tomás Santos
School of Technology and Management, University of Leiria and Oeste, Leiria, Portugal
2023123014@ulo.pt, 2023122551@ulo.pt
Emerging and Intelligent Technologies · Virtual and Augmented Reality · 2026/2027

***Abstract*—Formula Student (FS) tracks are marked with hundreds of colour-coded cones that teams set out by hand with tapes and trigonometry, a slow process with no check of the result, although driverless cars depend on the layout matching the plan. This paper reviews the state of the art for TrackSeed, a system in which one person sets out a track with augmented reality (AR) guidance on a smartphone while a cloud-hosted digital twin (DT) follows the placement live. The review covers AR for on-site layout, mobile AR tracking and alignment to the ground, real-time synchronisation and on-device inference, using studies selected for quantitative evidence and official documentation for tool capabilities. The literature shows that AR placement reaches centimetre-to-decimetre accuracy over small areas, with phones the least accurate devices, and that tracking error grows with the distance from the reference; no study has evaluated phone-based setout over a full track. Based on these findings, TrackSeed adopts two-point marker alignment with re-alignment along the track, WebSocket synchronisation with a resend queue, dashboard edits sent back to the phone as guidance, and on-device cone verification and car identification. Placement accuracy, setup time, synchronisation latency and car identification accuracy are proposed to validate it.**

***Index Terms*—augmented reality; digital twin; mobile AR tracking; spatial layout; Formula Student; real-time synchronisation; fine-grained vehicle classification**

I. Introduction

Formula Student (FS) is an international engineering competition in which university teams design, build and race a single-seat car; in the driverless (DV) class, the same car must complete the dynamic events autonomously. Every dynamic event is driven on a track that exists only as a pattern of colour-coded cones on an open area of tarmac. Under the 2026 driverless specification, the left border is marked with small blue cones, the right border with small yellow cones, entry and exit lanes with small orange cones, and big orange cones are placed before and after the start, finish and timekeeping lines, with at most 5 m between consecutive cones in the driving direction and shorter spacing in corners [1]. Until now, these layouts are reproduced many times per season by the team during test days: acceleration and skidpad layouts for vehicle tuning, and autocross and trackdrive circuits of a few hundred metres, which already need well over a hundred cones per lap.

These tracks are set out by hand: a plan drawn on paper or generated by software is transferred to the ground with measuring tapes and trigonometry, which needs several people, takes time away from testing and provides no check of the result: a misplaced or wrong-coloured cone is only corrected if someone notices it. For driverless cars, the consequences go beyond inconvenience. Their perception and mapping pipelines use the cones as the landmarks that define the track, and path planning and control are built on the map estimated from them [2]. A layout that differs from the intended one changes the problem the car is solving and makes results from different test days hard to compare. Results are also only comparable if each session is linked to the car that ran, which is today recorded by hand, if at all. Surveying instruments such as total stations and real-time kinematic (RTK) global navigation satellite system (GNSS) receivers reach centimetre accuracy [3], but they are dedicated equipment that student teams rarely own or are trained to use. None of these options links the plan, the placement on the field and a record of what was actually built.

Augmented reality (AR) combines virtual content with the user's view of the real world, registered in three dimensions and interactive in real time [4]. Mobile AR frameworks on consumer smartphones, such as Google's ARCore, estimate the pose of the device by combining visual feature points from the camera with inertial measurements [5]. This makes it possible to show each planned cone, with its colour, at its position on the ground through a smartphone, so that a single team member can set out the track alone. Compared with the conventional alternatives (a tape measure, or a 2D map showing the phone's GNSS position), AR guidance removes the measuring step, shows the instruction where the cone must be placed, and allows the same camera to verify the placement. Evidence from construction point layout shows both the potential and the main limitation: with a head-

mounted AR device registered to a paper marker, an accuracy of about 5 cm was reported, which became worse further from the marker [6]. Their indoor test area was much smaller than an FS test site, so this accuracy cannot be assumed to hold over a full track.

The proposed system, TrackSeed, is a joint project across several course units; this paper covers the contributions of two of them around a shared model of the track. Virtual and Augmented Reality (VAR) provides the AR guidance aligned to the real ground and a digital twin (DT) of the track: a live virtual model that is updated automatically with every cone confirmed by the phone, and whose edits on the dashboard are sent back to the phone as updated guidance. A full DT exchanges data automatically in both directions; because a person still has to move the physical cone, TrackSeed sits between a digital shadow and a full twin [7]. Emerging and Intelligent Technologies (EIT) provides the cloud platform that stores track plans and placement sessions, real-time synchronisation between the phone and a web dashboard, remote configuration of the app, and on-device intelligence that checks each placed cone with the camera. In this first version, one person places the cones with one phone; collaborative placement with several phones is left for future work.

This paper reviews the state of the art that TrackSeed builds on, with three objectives: (i) to establish what accuracy mobile AR achieves for on-site layout and guidance, and which alignment approaches limit drift over large outdoor areas; (ii) to compare architectures for keeping a DT synchronised in real time between mobile and web clients; and (iii) to derive and justify the technology choices and validation plan of the proposal. Section II reviews the literature by theme, Section III analyses the candidate technologies, Section IV presents the proposed solution and its validation plan, Section V discusses feasibility and limitations, and Section VI concludes.

A. Scope and literature search

The literature was searched in October 2026 in Google Scholar and in publishers' digital libraries, assisted by an AI tool (see the AI use declaration) and by following the references and citations of the most relevant studies. Search strings were built from the main concepts of the problem (AR, on-site layout, mobile tracking accuracy and digital twins) and their synonyms, for example ("augmented reality" OR "mixed reality") AND ("layout" OR "setout" OR "stakeout") AND ("accuracy" OR "error"), and ("ARCore" OR "mobile AR") AND ("drift" OR "accuracy") AND "outdoor". A source was kept only if it was directly relevant to the problem, reported evidence that could be compared, came from a credible venue, was recent enough to reflect current devices, and helped compare alternatives or justify a design decision; peer-reviewed papers published from 2015 onwards were preferred. Foundational works were kept for definitions, and official documentation and the FS rules were used only for tool capabilities and domain requirements, not as evidence of performance. Since almost no studies address track setout in motorsport, the review relies mainly on adjacent domains, construction layout in particular, whose tasks, distances and accuracy requirements differ from those of an FS test site.

II. State of the Art

A. AR for on-site layout and spatial guidance

Construction layout, where workers mark the positions of building elements from a design model, is the task closest to setting out a track, and it is the domain where AR placement has been measured most directly. The studies found differ mainly in the device used and in the size and type of the test area.

Head-mounted AR in small indoor areas. Chalhoub and Ayer asked industry practitioners to lay out electrical points with a HoloLens registered to a paper marker, and measured each point with a laser distance meter [6]. Content was displayed about 5 cm from its design position, and the error was smaller near the marker. Showing more content, or content of different types, also reduced accuracy in elevation. The test was carried out indoors, over an area much smaller than a track.

Head-mounted, handheld and GNSS-assisted AR outdoors. Kim and Olsen compared four devices in an outdoor, partly shaded simulation of underground utility placement, with 32 construction students placing survey prisms where they perceived each virtual pipe and a robotic total station recording the positions [8]. The mean error was 3.28 in (about 8.3 cm) with a HoloLens 2, 3.52 in (8.9 cm) with Trimble SiteVision, which adds a GNSS receiver, 3.59 in (9.1 cm) with an iPad Pro and 6.26 in (15.9 cm) with an iPhone 13 Pro Max, with eight observations per device and pipe. An earlier report on the same scenario also found the HoloLens to be the most accurate device and concluded that the devices did not meet the strict tolerances of construction work [9].

The two groups of studies agree that AR placement reaches centimetre-to-decimetre accuracy over small areas, that head-mounted devices are more accurate than handheld ones, and that accuracy depends on the distance from the reference used to register the content. They differ by a factor of about two in the error they report, which can be explained by the indoor versus outdoor setting and by the participants' experience, so the values cannot be compared directly. For TrackSeed, the most relevant figure is the phone result: an error of about 16 cm outdoors is within the provisional tolerance of 25 cm assumed for an FS track (Section III), but it already uses most of that margin and was obtained over a much smaller area than a track, before any drift over distance. The literature therefore supports AR guidance as feasible, but not as accurate enough by default; the accuracy of phone-based setout over a full track is the main hypothesis TrackSeed has to test.

B. Mobile AR tracking accuracy and alignment to the ground

Placement accuracy over a full track depends on how well the phone keeps its position as the user walks, and on how the virtual plan is aligned to the real ground. Four approaches appear in the literature: relying on the phone's own visual-inertial tracking after an initial alignment, registering the

TABLE I
COMPARISON OF THE REVIEWED LITERATURE

Study	Approach	Context and technology	Evidence / results	Limitations	Relevance to TrackSeed
Chalhoub and Ayer [6]	Head-mounted AR layout	Indoor electrical point layout; HoloLens with paper marker	About 5 cm error; smaller near the marker	Indoor, small area; no time comparison	Accuracy depends on distance from the reference
Kim and Olsen [8], [9]	Handheld, head-mounted and GNSS-assisted AR	Outdoor utility placement; 32 students; total-station ground truth	Mean error 8.3 cm (HoloLens 2) to 15.9 cm (iPhone)	Small area; 8 observations per device and pipe; students	Phone error already uses most of the 25 cm FS tolerance
Feigl et al. [10]	Visual-inertial tracking	ARCore, ARKit, HoloLens in a 1,600 m^2 hall; optical ground truth	About 17 m error per 120 m; scale error up to 14.4 cm/m	Dynamic indoor industrial setting	Re-alignment is needed over long distances
Scargill et al. [11]	Visual-inertial tracking	Three phones, more than 200 indoor scenes	Drift of a few cm; 4.8 cm (ARCore) vs 25.1 cm (ARKit) after walking away	Room scale; preprint	Low texture and light increase drift (asphalt risk)
Nunes et al. [12]	Multi-marker registration	Large indoor industrial space; HoloLens with several markers	Two markers significantly reduce angular error; more points improve accuracy	Indoor; head-mounted device	Supports two-point alignment at the start
Jiang et al. [3]	Smartphone RTK GNSS	Unmodified phones; static and kinematic tests in open sky	1.1–2.1 cm static, about 4 cm kinematic RMS error	Algorithm study; no AR; open sky only	Absolute anchoring without drift
Ajayi et al. [13]	Pub/sub vs client–server	MQTT vs WebSocket for IoT telemetry	MQTT: higher throughput, fewer resources; WebSocket: lower latency	Single controlled test; telemetry data	Protocol choice for phone–dashboard sync
Chen and Ran [14], Ignatov et al. [15]	Edge vs cloud inference	Review of edge deep learning; smartphone AI benchmark	On-device avoids network latency and privacy risks; mobile NPUs approach recent desktop GPUs	Review and benchmark, not this task	Run the cone check on the phone
Vödisch et al. [16], Dhall et al. [17]	FS cone detection	Shared cone dataset; real-time detection on a driverless car	Annotated FS cone data; real-time detection on embedded hardware	Car-mounted cameras, not handheld	Training data and model baseline
Yang et al. [18], Yosinski et al. [19], Sandler et al. [20]	Fine-grained car classification and transfer learning	Large car dataset; transfer of pretrained features; mobile CNN	Accuracy depends on viewpoint; pretrained features help small datasets; 72% ImageNet top-1 with 3.4M parameters	Public car models, not a team's own cars	Self-made dataset with pretrained mobile backbone

content repeatedly against several references, localising the phone against a visual map, and using satellite positioning.

Visual-inertial tracking alone. ARCore estimates the phone's pose from camera features combined with inertial measurements [5]. In a 1,600 m^2 industrial hall with walking distances of 60 m, Feigl et al. found that ARCore, ARKit and HoloLens accumulated an average error of about 17 m per 120 m walked, with a scale error of up to 14.4 cm/m that grew roughly in proportion to the path length [10]. At room scale, Scargill et al. measured the drift of virtual objects on three phones in more than 200 environments and found errors of a few centimetres that depended on what the user did between views: when the user walked away and returned, the mean drift was 4.8 cm with ARCore and 25.1 cm with ARKit [11]. Drift also increased in dim scenes and in scenes with little texture or low contrast.

Registration against several references. Nunes et al. proposed a calibration model that registers AR content in a large indoor industrial space with several markers, tested on a HoloLens; they report that single small markers are numerically unstable, that using two markers significantly reduces the angular discrepancy between real and virtual points, and that accuracy improves as more calibration points are added [12].

Visual positioning. The ARCore Geospatial API localises the phone against imagery from Google Street View and is described by Google as more accurate than GPS alone, but the documentation does not give accuracy figures and the service only works where Street View coverage exists [21]. Skabek et al. compared it with other AR localisation platforms for indoor and outdoor campus navigation [22], a use case whose accuracy needs are far looser than cone placement. No independent evaluation at the centimetre level was found.

Satellite positioning. Standard phone GNSS is not accurate enough for cone placement, but RTK corrections change this: Jiang et al. reached root-mean-square errors of 1.1–2.1 cm in a static test and about 4 cm in a kinematic test with unmodified smartphones in open-sky conditions [3]. In the outdoor study of Kim and Olsen, the GNSS-assisted SiteVision was more accurate than both handheld devices but not better than the HoloLens [8].

The two tracking studies appear to disagree, with errors of metres in one and centimetres in the other, but the difference is explained by scale: the error of visual-inertial tracking grows with the distance walked [10], and a room-scale study cannot

reveal it. Both agree that the environment matters, and Scargill et al.'s finding that low texture increases drift is a risk for the uniform asphalt of a test site, although no study measured it on tarmac. Registration against several references addresses the angular error of a single marker [12], but it has only been tested indoors with a head-mounted device. The trade-off is clear: visual tracking is free and works anywhere but drifts over distance; visual positioning depends on map coverage and lacks published accuracy; RTK GNSS gives centimetre accuracy without drift but needs a receiver, a correction source and an open sky, which an FS test site usually provides. For TrackSeed, these findings mean that a single alignment at the start line cannot keep a track of several hundred metres within tolerance, and that the plan must be re-aligned at reference points along the track or anchored to absolute positions.

C. Real-time platforms and synchronisation

Live updates between the phone, the cloud and the dashboard can follow a client–server model, typically WebSocket connections, or a publish–subscribe model such as MQTT. Ajayi et al. compared both for streaming IoT telemetry and found that MQTT achieved higher throughput and used fewer system resources, while WebSocket had slightly lower round-trip times [13]. The evidence found for this theme is limited to a single controlled comparison with telemetry data, so it shows a trade-off rather than a general ranking. For TrackSeed, the volume of data is small (one message per confirmed cone) and the receiving end is a web dashboard, which favours WebSocket's lower latency and simpler deployment, without a separate broker, over MQTT's efficiency at scale; this is assessed further in Section III.

D. Edge AI for cone verification and car identification

TrackSeed runs two vision models, one that checks each placed cone and one that identifies the team car being used; each of which can run in the cloud, on the phone or in a hybrid of both. The trade-off is that sending camera frames to the cloud gives access to more computing power but adds latency and privacy risks while using uplink bandwidth, while on-device inference avoids these costs but is limited by the device's computing power and memory [14]. For the cone check, FS cone detection already runs in real time on embedded hardware [17], and the FSOCO dataset provides annotated FS cone images [16]. Frameworks such as TensorFlow Lite (now LiteRT) provide on-device inference and mobile hardware has narrowed the gap: in a benchmark of smartphone AI accelerators, the newest mobile neural processing units were approaching the performance of CUDA-compatible Nvidia graphics cards released not long before [15].

No public dataset contains a team's own cars, so TrackSeed must use a small dataset made by the authors. Fine-grained car classification is harder than generic recognition, because classes differ in small details and accuracy depends on the viewpoint [18]; TrackSeed has only a few classes, but cars from the same team often look alike. Its images also all come from one site, so training and testing on frames from the same day would overestimate accuracy. Features learned on large datasets transfer well to small ones [19], and mobile networks such as MobileNetV2 keep the model small enough for a phone [20]. TrackSeed therefore fine-tunes a pretrained mobile network on photos of each car from all viewpoints, taken on several test days.

E. Comparative synthesis

Table I summarises the studies reviewed. Four findings stand out. First, every study that measured position over distance found that error grows with the distance from the reference, whether that reference is a marker [6] or the starting point of tracking [10]; the studies disagree on magnitude only because they test different scales. Second, almost all accuracy evidence comes from areas of a few metres to tens of metres, while an FS track covers hundreds; the only large-scale study measured errors far above any useful tolerance without re-alignment [10]. Third, phones are consistently less accurate than head-mounted devices [8], yet phones are the only devices FS teams have available. Fourth, pretrained mobile networks make on-device car classification feasible with little data [19], [20], but the reported accuracies come from large public datasets, not from a few similar cars photographed at one site. What cannot be concluded from the literature is whether phone-based AR, combined with re-alignment or RTK positioning, keeps a full track within tolerance: no study has tested this. No work was found that combines placement guidance, automatic verification of each placed object, identification of the vehicle under test and a live two-way digital twin, which is the gap TrackSeed addresses.

III. Technological Analysis

The analysis starts from five requirements derived from Section I: (R1) each cone within a provisional tolerance of 25 cm, 5% of the maximum cone spacing of 5 m [1], of its planned position over tracks of 100 m or more; (R2) operation by one person with one Android phone; (R3) operation on a field without reliable mobile coverage; (R4) a twin that updates within about one second and accepts edits in both directions; and (R5) development effort compatible with one semester.

A. Aligning the track plan to the ground

Alignment is driven by R1 and R2, against which Table II compares the options. The tape method fails R2, since it needs two or more people. A single marker at the start line is the simplest option, but it fixes the heading of the whole plan from a marker less than a metre wide: a heading error of only 1° moves a cone 200 m away by about 3.5 m, and visual-inertial drift adds to this as the user walks [10]. Aligning on two markers placed far apart fixes position and heading together, and re-aligning at further markers along the track limits the drift to the distance between markers. Both steps are supported by the literature: Nunes et al. found that two markers significantly reduce the angular error of one [12], and error is smallest near the reference [6]. The Geospatial API is rejected as the main method because no evidence shows it reaches

TABLE II
APPROACHES FOR ALIGNING THE TRACK PLAN TO THE REAL GROUND

Approach	Reported accuracy	Range and drift	Cost	Setup effort	People	Sources
Tape and trigonometry (baseline)	Not reported; depends on the operator	No drift; errors accumulate per measurement	Very low	High	Two or more	Team practice
Single start marker	About 5 cm near the marker, worse further away	Heading error grows with distance; drift up to 17 m per 120 m	Low	Low	One	[6], [10]
Two-point alignment + re-alignment markers	Two markers reduce angular error [12]; not tested on phones over a track	Drift limited to the distance between markers	Low	Medium (markers placed once per site)	One	[12], [6]
Geospatial API (visual positioning)	No published centimetre-level figures	Needs Street View coverage	Free	Low	One	[21], [22]
Smartphone RTK GNSS	1.1–4 cm RMS in open sky	No drift; needs sky view and corrections	Medium	Medium	One	[3]

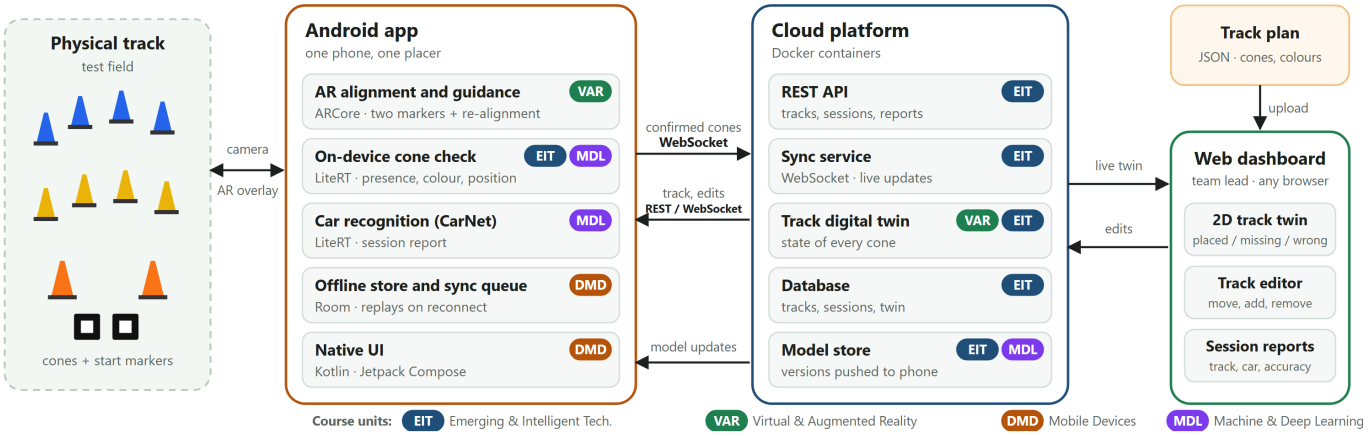


Fig. 1. Conceptual architecture of TrackSeed. The phone guides and verifies the placement on the physical track and sends each confirmed cone to the cloud over WebSocket; the cloud keeps the track twin, which the dashboard displays live, and returns track edits and model updates to the phone. Tags show the course unit responsible for each part.

the required accuracy [21]. RTK GNSS reaches centimetre accuracy without drift [3], but needs a correction service and suitable hardware; it is kept as an optional extension, to be added only if field tests show that marker alignment cannot meet R1. TrackSeed therefore adopts two-point marker alignment with re-alignment markers. Two decisions remain open: the spacing between markers, to be set from field tests, and how the markers are positioned, which only needs to be done once per site with the team's usual method or an RTK receiver.

B. Mobile AR platform

The platform is driven by R2, which fixes the device to one Android phone, and by R1, through the quality of tracking. Native Android with ARCore, Unity with AR Foundation and native iOS with ARKit were considered. Native Android in Kotlin, the most direct option for one device within one semester (R5), gives direct access to the ARCore session and its camera frames, which the on-device cone check needs. Unity would add cross-platform reach that R2 does not require, at the cost of an extra runtime layer between the camera and the model. For R1, tracking quality does not rule out either platform, but in the condition Scargill et al. tested that is most similar to walking between cones, walking away and returning, ARCore drifted less than ARKit (4.8 vs 25.1 cm) [11], which favours Android. The compromise is that the app runs only on ARCore-supported Android phones; other users reach the twin through the web dashboard.

C. Platform and real-time communication

Communication is driven by R4, since each confirmed cone and each edit must reach the other side within about one second, and by R3, since no confirmation may be lost when coverage drops. Table III compares the options. REST polling would make R4 depend on the polling interval. TrackSeed sends one small message per confirmed cone, and the dashboard runs in a browser; in this setting, WebSocket's lower latency [13] and the absence of a separate broker outweigh MQTT's higher throughput and lower resource use, which matter for many constrained devices rather than one phone. MQTT can also reach a browser over WebSocket, so browser support does not separate the two. A managed real-time database would provide offline synchronisation out of the box [23], but it ties the platform to one provider and gives

TABLE III
REAL-TIME COMMUNICATION OPTIONS

Option	Latency	Offline support	Scale	Effort
REST polling	Limited by the polling interval	Client must queue writes	Many redundant requests	Low
WebSocket	Lower than MQTT [13]	Client must queue writes	One connection per client	Medium; native in browsers
MQTT	Slightly higher [13]	Session state kept by the protocol [24]	Higher throughput, fewer resources [13]	Medium; needs a broker
Managed real-time DB	Not compared	Local cache and queued writes [23]	Managed by provider	Low; vendor lock-in

less control over the twin's data model. TrackSeed therefore uses WebSocket for live updates and REST for uploading and downloading tracks and reports. So that no confirmation is lost if the connection drops, the phone also stores every confirmed cone locally and resends the queued messages when the connection returns, the same pattern a managed database applies internally. With no connection at all, the phone keeps guiding and verifying from the plan downloaded beforehand, and the dashboard catches up when the queue is sent; the live view therefore needs mobile coverage or a local Wi-Fi hotspot, but placement does not. The services run in Docker containers, and tracks use an open JSON schema so that plans from other FS tools can be imported.

D. Digital twin and on-device intelligence

The twin stores, for every cone, its planned position and colour and a state: planned, placed, verified, wrong colour or missing, with the measured offset. Because cones lie on a plane, a 2D top-down map is enough for the dashboard; a 3D view is optional. Edits made on the dashboard go through the sync service to the phone and change the guidance; the placer then moves the cone, so the loop back to the physical track goes through a person. The cone check measures the offset of every placed cone from its guided position. Because this offset is measured with the same AR tracking that guides the placement, it catches wrong colours, missing cones and placement mistakes, but not errors caused by tracking drift, which only independent measurements can reveal (Section IV-B). Where the check runs affects R4: on the phone, only a small confirmation message has to travel over the network, instead of camera frames [14]. Following Section II-D, the check therefore runs on the phone with a compressed model in LiteRT, while the cloud stores training data and pushes new model versions, which also provides remote configuration. A second on-device model, developed in the Machine and Deep Learning course, recognises the team car for the session report. Selecting the car from a list or scanning a QR tag on it would be simpler and is kept as a fallback, but a manual entry depends on the operator remembering it, which is how the record is lost today, and tags must be fitted and kept visible on every car; recognition from the camera needs neither. Table IV summarises how the evidence of Section II leads to each decision.

TABLE IV
FROM RESEARCH FINDINGS TO DESIGN DECISIONS

Evidence	Implication	Decision	Req.
AR accuracy degrades with distance from the marker [6]	One marker cannot keep a 200 m track within tolerance	Two-point alignment at the start	R1, R2
Tracking error grows with distance walked [10]	Phone tracking alone cannot hold a full lap	Re-alignment markers; RTK as an option [3]	R1
ARCore drifted less than ARKit after walking away [11]	Tracking quality does not penalise Android	Native Android, the single device of R2	R1, R2, R5
WebSocket has lower latency, MQTT higher throughput [13]	One small message per cone; the client is a browser	WebSocket, with a resend queue on the phone	R3, R4
On-device inference avoids network latency and privacy costs [14]	Only a small message per cone needs the network	Cone check on the phone; models updated remotely	R1, R4

IV. PROJECT PROPOSAL

A. Solution and conceptual architecture

Before a test day, the team lead uploads a track plan, a JSON file with the position and colour of every cone, through the web dashboard. On the field, one team member opens the track on an ARCore-supported Android phone, aligns it by scanning two printed markers at the start line, and walks the track while the app shows the position and colour of each cone in AR. When a cone is put down, the on-device model checks that it is present, has the right colour and lies within tolerance of its guided position, and the phone sends the result to the cloud. The team lead follows the twin live on the dashboard and can move, add or remove cones, which immediately updates the guidance on the phone. Before the run, the user points the phone at the car, the on-device classifier identifies which team car is being used, and the app produces a session report with the track, the car, the setup time and the placement accuracy.

Fig. 1 shows the four components and the course unit behind each part, including two further units of the joint project that are outside the scope of this paper: mobile application development (DMD) and Machine and Deep Learning (MDL). The Android app (DMD) runs the AR alignment and guidance (VAR), the cone check and car recognition (EIT, MDL) and the offline store. The cloud platform (EIT) holds the REST API, the WebSocket sync service, the database with the twin state, the self-made car dataset and the model store. The web dashboard (EIT, VAR) shows and edits the twin. Tracking and inference run on the phone; storage, synchronisation, validation of uploaded tracks and model training run in the cloud. The system depends on an ARCore-supported Android phone, printed markers, a host for the containers, the FSOCO dataset [16] plus the team's own images for the cone check,

TABLE V
PLANNED EVALUATION CRITERIA

Criterion	**Metric**	**Proposed test**	**Success indicator**
Placement accuracy (H1)	Error per cone (cm)	Set out a skidpad and an autocross section; measure each cone against reference points	Median $\leq$15 cm; no cone over 25 cm
Setup efficiency (H2)	Setup time (min)	One person with TrackSeed vs the team's tape method on the same track; elapsed time and person-minutes	At least 30% faster
Twin synchronisation (H3)	End-to-end latency (ms)	Timestamps from cone confirmation to dashboard display, on Wi-Fi and mobile data	Median $\leq$1 s
Car identification (H4)	Accuracy and macro F1	Self-made dataset split by test day; held-out days only, on the phone	$\geq$95% accuracy; inference $\leq$100 ms

and the self-made car dataset. Several phones placing cones at once, iOS, and the Geospatial API as the main alignment method are out of scope. The preliminary plan is: October, track format, AR alignment on a skidpad and the start of car image collection, continued on every test day; November, cone verification, car classifier, cloud platform and synchronisation; December, dashboard twin and full autocross tests; January, validation and report. The marker spacing and the model architecture remain provisional.

B. Preliminary validation

Table V lists the four criteria, each linked to a hypothesis. H1: phone-based setout with re-alignment keeps cones within the tolerance over a full track, the question the literature leaves open. H2: one person with TrackSeed sets out a track faster than the team's usual tape method. H3: the twin reflects each confirmed cone on the dashboard in about one second. H4: a pretrained mobile network fine-tuned on the self-made dataset identifies the team car on test days not seen in training. The tests will use a skidpad, whose geometry is fixed by the FS rules [1], and an autocross section on the team's test site, measuring each cone against tape measurements from surveyed reference points or a total station, if one is available; the reference measurements must be accurate to a few centimetres, well below the 25 cm tolerance. Each test is repeated on at least three test days, and setup time is reported both as elapsed time and in person-minutes, since the tape method needs several people. Unlike the reviewed studies, which tested small areas with student participants [8], these tests cover a full track in its real environment. The success indicators are provisional and will be reviewed after the first field test.

V. DISCUSSION

The proposal is feasible within one semester because it combines existing, documented technologies (ARCore, LiteRT, WebSocket) rather than developing new tracking or networking methods, and because the single-placer scope avoids the problem of aligning several phones. The main technical risk is accuracy over distance: the evidence shows error growing with the distance from the reference [6], [10], and no study measured phone AR on asphalt, a low-texture surface that is likely to increase drift [11]. Outdoor lighting, glare on the screen and wet weather during winter tests may also affect both tracking and the cone check, whose training data come from car-mounted cameras [16]. For car identification, the main risks are the size of the self-made dataset and the similarity between the team's cars; a model that relies on the background or the light of one day rather than on the car would pass a random split and fail on a new day, which is why the test set is split by day. The user's safety is a further concern, as attention is on the screen while walking; the app is meant to be used only on a closed track.

The evidence itself has limits. Most accuracy studies come from construction, indoors or over small areas, with few participants [6], [8]; one tracking study is a preprint [11]; and the protocol comparison is a single controlled test [13]. The review changed the proposal in two ways: a single start marker was replaced by two-point alignment with re-alignment markers, and the Geospatial API was moved from a candidate to a rejected option. The twin is also limited: dashboard edits reach the phone automatically, but a person must move the physical cone. Unity, iOS and dedicated surveying instruments were rejected for this version; collaborative placement with shared anchors and an integrated RTK receiver are left for future work. The expected contribution is a justified integration and evaluation of existing technologies for a task that has not been studied, not a new tracking method.

VI. CONCLUSIONS

This paper reviewed the evidence behind TrackSeed, an AR system for setting out FS tracks with a live digital twin. The literature shows that AR placement reaches centimetre-to-decimetre accuracy over small areas, that phones are the least accurate devices, and that tracking error grows with the distance from the reference; no study has evaluated phone-based setout over a full track. These findings led to two-point marker alignment with re-alignment, WebSocket synchronisation with a resend queue, a live twin whose edits reach the phone as guidance, and on-device cone verification and car identification. The next steps are to implement AR alignment on a skidpad, measure the drift on the team's test site to set the marker spacing, and then validate placement accuracy, setup time, synchronisation latency and car identification on a full track.

AI USE DECLARATION

The authors used Claude (Anthropic, Claude Opus 5.5) to analyse the assignment brief and lecture guidelines, propose the paper structure, search for and summarise literature, draft the text and tables of all sections, and produce the architecture diagram in Fig. 1. All outputs were reviewed and edited by

the authors, and references and technical claims were checked against their sources. The authors remain responsible for the submitted work.

REFERENCES

[1] Formula Student Germany, "Formula student driverless specification 2026," Version 1.1, May 2026, accessed: Oct. 22, 2026. [Online]. Available: https://www.formulastudent.de/fileadmin/user_upload/all/2026/rules/FS_Driverless_Specification_2026_v1.1.pdf

[2] J. Kabzan, M. de la Iglesia Valls, V. Reijgwart *et al.*, "AMZ driverless: The full autonomous racing system," *Journal of Field Robotics*, vol. 37, no. 7, pp. 1267–1294, 2020.

[3] Y. Jiang, Y. Gao, W. Ding, F. Liu, and Y. Gao, "An improved ambiguity resolution algorithm for smartphone RTK positioning," *Sensors*, vol. 23, no. 11, p. 5292, 2023.

[4] R. T. Azuma, "A survey of augmented reality," *Presence: Teleoperators and Virtual Environments*, vol. 6, no. 4, pp. 355–385, 1997.

[5] Google, "ARCore fundamental concepts," Google for Developers, accessed: Oct. 22, 2026. [Online]. Available: https://developers.google.com/ar/develop/fundamentals

[6] J. Chalhoub and S. K. Ayer, "Effect of varying task attributes on augmented reality aided point layout," *Journal of Information Technology in Construction (ITcon)*, vol. 24, pp. 95–111, 2019. [Online]. Available: https://www.itcon.org/2019/6

[7] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, "Digital twin in manufacturing: A categorical literature review and classification," *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.

[8] J. Kim and D. Olsen, "Enhancing construction inspection with augmented reality: An analysis of accuracy and observation of its application in training," in *Proc. 42nd Int. Symp. Automation and Robotics in Construction (ISARC)*, 2025. [Online]. Available: https://www.iaarc.org/publications/fulltext/074_Enhancing_Construction_Inspection_with_Augmented_Reality__An_Analysis_of_Accuracy_and_Observation_of_its_Application_in_Training.pdf

[9] ——, "Enhancing construction industry training through augmented reality: A study of student inspectors," in *Human Factors in Design, Engineering, and Computing (AHFE International)*, vol. 159, 2024.

[10] T. Feigl, A. Porada, S. Steiner, C. Löffler, C. Mutschler, and M. Philippsen, "Localization limitations of ARCore, ARKit, and Hololens in dynamic large-scale industry environments," in *Proc. 15th Int. Joint Conf. Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP), Vol. 1: GRAPP*, 2020, pp. 307–318.

[11] T. Scargill, J. Chen, and M. Gorlatova, "Here to stay: Measuring hologram stability in markerless smartphone augmented reality," arXiv preprint arXiv:2109.14757, 2021.

[12] J. S. Nunes, F. B. C. Almeida, L. S. V. Silva, V. M. S. O. Santos, A. A. B. Santos, V. de Senna, and I. Winkler, "Three-dimensional coordinate calibration models for augmented reality applications in indoor industrial environments," *Applied Sciences*, vol. 13, no. 23, p. 12548, 2023.

[13] O. Ajayi, A. Bagula, J. Bode, and M. Damon, "A comparison of publish-subscribe and client-server models for streaming IoT telemetry data," in *Emerging Technologies for Developing Countries*, ser. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering. Cham: Springer, 2023, pp. 129–139.

[14] J. Chen and X. Ran, "Deep learning with edge computing: A review," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1655–1674, 2019.

[15] A. Ignatov, R. Timofte, A. Kulik, S. Yang, K. Wang, F. Baum, M. Wu, L. Xu, and L. V. Gool, "AI benchmark: All about deep learning on smartphones in 2019," in *Proc. IEEE/CVF Int. Conf. Computer Vision Workshops (ICCVW)*, 2019, pp. 3617–3635.

[16] N. Vödisch, D. Dodel, and M. Schötz, "FSOCO: The formula student objects in context dataset," *SAE International Journal of Connected and Automated Vehicles*, vol. 5, no. 1, 2022.

[17] A. Dhall, D. Dai, and L. V. Gool, "Real-time 3D traffic cone detection for autonomous driving," in *Proc. IEEE Intelligent Vehicles Symposium (IV)*, 2019, pp. 494–501.

[18] L. Yang, P. Luo, C. C. Loy, and X. Tang, "A large-scale car dataset for fine-grained categorization and verification," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3973–3981.

[19] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014, pp. 3320–3328.

[20] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[21] Google, "ARCore geospatial API," Google for Developers, accessed: Oct. 22, 2026. [Online]. Available: https://developers.google.com/ar/develop/geospatial

[22] K. Skabek, D. Rola, and W. Zamarski, "Combining indoor and outdoor positioning for navigation in AR environments," *Computer Assisted Methods in Engineering and Science*, vol. 33, no. 1, pp. 7–25, 2026.

[23] Google, "Enabling offline capabilities on Android (Firebase realtime database)," Firebase Documentation, accessed: Oct. 22, 2026. [Online]. Available: https://firebase.google.com/docs/database/android/offline-capabilities

[24] OASIS, "MQTT version 5.0," OASIS Standard, Mar. 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html